COMP 307

Course Team Project

Multiple Due Dates (see below)

Introduction

This is your COMP 307 end-of-term project. Your team size must be 3 or 4 students.

This term's project has 3 development options:

- SOCS Booking Application Competition Project (the **SOCS Website Competition**), or
- SOCS Booking Application Regular Project, or
- Do-it-yourself Theme-based Project.

Team Organization and Project Type Selection

You have already registered your team and selected the project type in Assignment #2. If you have not yet done this, please speak to the professor (there will be a point deduction for being late).

You are not permitted to switch the application program after you have started it; you can switch between submitting your project to the competition and removing it from the competition. You will need to update the team registration form to indicate this change. Speak with the professor if needed.

You should start meeting with your teammates at least weekly to plan your project. The key to working with people is to meet with them regularly. I suggest setting a day and time, like every Friday at 4 PM, and make it a habit to meet. The entire team must be present. Keep one another accountable. It will be so much more enjoyable. Each week, set a task that should be completed before the next meeting, and progress this way throughout the project. Speak with the professor early if you are having problems in your team.

This project gives you a lot of flexibility in design and the technology stack. I will give you general guidelines to follow, but otherwise, you have a lot of freedom in what and how you build the application. You are permitted to use technology we did not cover in class, as long as you respect these conditions: (1) the technology you select should not make it difficult for the TA to grade your work, and (2) for the competition, submissions must run on Mimi. Mimi can support both Apache and Node.js applications. It can support both SQL and MongoDB databases. You can use any frontend.

Below, I will give general project guidelines. You will fill in the gaps with your own ideas. Discuss with your team. Talk to me if you need additional guidance. Talk to the TAs for help with programming. You can also email help@cs.mcgill.ca for all Mimi-related support questions (they are expecting you).

SOCS Booking Application Project

Purpose: The professors of McGill would like to have a booking application. We want students to book appointments with us rather than emailing us to request one. They would use this tool instead. We would like to set up office hours so that students can reserve a time slot. We also want this to work for teaching assistants. We also need a way to schedule group meetings that uses a heatmap to determine the best time. We would also like these appointments to appear on our Google or Outlook calendars either automatically or through an export/import file. This website needs a good name.

Details: Below is a list of basic features the application must have. In addition, there will be a list of optional bonus features. If you have a group of three on your team, you should probably not use any of the bonus features unless you want to. If you are in a group of four students, then you should probably consider doing one small bonus feature.

Required Features:

- Need an initial landing page that introduces the booking application, displaying its name.
- Registration and basic operation instructions
    - Only people with a McGill email can register. Only registered users can create and reserve booking slots.
    - Only @mcgill.ca emails can create and manage booking slots. We will call them owners.
        - The owner can create booking slots. Slots start out private and visible only to the owner. The owner can activate a slot, making it public to everyone. Once a slot is activated, other users can reserve that slot for an appointment.
        - Owner can generate an invitation URL (to put in slides or emails). This URL will take the user directly to a Booking Page that shows only the owners' activated slots. This will ask the user to log in.
        - Owner can delete the booking slot. If someone had reserved the slot that was deleted, they would receive a notification email (mailto:).
        - Owner can view all slots and who booked a slot.
        - Owner can send an email to the booked person (using mailto: not a mail server)
    - Only @mail.mcgill.ca and @mcgill.ca can book an active slot
        - Implement a listing feature to see all registered @mcgill.ca people who have active booking slots. Select the owner you are interested in, and then see their available slots.
        - Users can book a slot.
        - Users can see all the slots they have booked
        - Users can delete a booking (making the slot available for someone else). The slot owner receives a notification email (mailto:).
        - Users can message the owner of the booking slot (using mailto:)
- After Login
    - Users see a dashboard listing their appointments and providing the following options:
        - They can email (mailto:) the owner of a slot (selected or open slot)
        - They can delete appointments.
        - They can book additional appointments.
        - They can logout
- About booking appointments
    - Owners create the slots. Those who want to meet the owner can reserve an available slot. Let us call these people users. There are three ways to create booking slots, as shown below under Types 1-3.
    - Type 1 – Request a meeting
        - A user sends a message to an owner requesting a meeting. The owner receives a notification email, and it appears in the owner's dashboard list of requested appointments, but it is not approved. From this list, a request can be accepted or declined. If accepted, the request creates a new booking slot and sends an

email to the user. The booking appears on both the user's and the owner's dashboards as an appointment.

- Type 2 – Group meeting
  - Calendar Method:
    - This method assumes a booking owner and the invited users.
    - The owner of this booking method picks specific dates and times in a day, starting from a specific date and ending at another date, indicating the times the owner is available. For example, they might indicate they are available on Monday from 2 PM to 3 PM, Monday from 5 PM to 6 PM, and Tuesday from 9 AM to 10 AM. Users can only select from the limited dates/times defined by the owner. The program counts how many times a slot was selected. Users can select multiple slots.
    - The owner can then select any period they wish, consulting the count for each date/time. The selection operation creates a shared booking slot. The owner can define this meeting as a one-time or recurring event. For example, the owner selects Monday at 9 AM for the meeting and would like to repeat this date/time for 5 consecutive weeks. A notification email is sent to the owner, and the booking appears on both the user's and the owner's dashboards as an appointment.
- Type 3 – Recurring Office Hours
  - This uses the Calendar Method, but there are no invited users, and no procedure for selecting the best meeting time. All the defined slots are available for reservation to the users of this website.
  - The owner defines one or more slots in a week that can be reserved, like in the Calendar Method, and the owner defines the number of weeks these slots will be available. For example, a professor has office hours on Monday at 10 AM and on Tuesday at 10 AM and would like those slots to repeat throughout the semester. Users can then reserve a slot, and a notification email is sent to the owner, and the booking appears on both the user's and the owner's dashboards as an appointment.

Optional Bonus Features:

- Additional Type 2 group meeting method:
  - This means that the website will support both the Heatmap and Calendar methods of creating an appointment. The owner will be asked to select the booking method they want to use.
  - Heatmap Method:
    - The booking owner selects a starting and ending date only. Each day in this period is displayed in 30-minute increments from 6 AM to 11 PM. An invitation is then sent to the invited users to select the times that work best for them. The user selects each 30-minute interval to indicate their availability. They can select as many 30-minute intervals as they want. The app counts the number of intervals each person selected. If Bonnie and Jane select the same interval, then it will record the count of 2 for that interval.
    - The owner of the heatmap then selects the best time given the distribution. The selection operation creates a shared booking slot. A notification email is sent to

the invited people, and the appointment appears on everyone's dashboard.

- Export to Calendar feature on Dashboard:
  - Export appointments to Google and Outlook calendars. Better yet, these calendars are automatically populated. This auto-calendar population would need a configuration option on the dashboard. Each user and owner can export their own appointments to their Google or Outlook calendar.
- Make the website secure.
  - Encrypted packets
  - Protected private webpages
  - Data privacy measures
- McGill Tinder (or a better name)
  - This is a tool that users can take advantage of to find teammates for a project.
  - A user (@mail.mcgill.ca) creates a team request entry. From the dashboard, other users can find this request entry. They can email the creator of this request. They can join the team. They can remove themselves from the team. The creator of the request can remove team members from the request. The creator can delete the request.
  - Any user can create a request. Each user can create multiple requests.
  - The request should display important information, like: the creator of the request, the course number, the team name, and an advertisement to attract people to the team.

SOCS Booking Application Competition Project

The specification for the Competition project is the same as the regular booking application project, except in four important ways:

1. Your website needs to run very well and look professional.
2. You must implement the export calendar bonus feature.
3. Mimi only supports Apache and Node.js applications using SQL or MongoDB databases on the backend, but you have no restrictions on the frontend framework/languages, etc.
4. Optionally, you will stand out if you implement 1 or more additional bonus features.

Do-it-yourself Theme-based Project

If you do not want to build a booking application or you are not motivated by the competition, then you can select the Do-it-yourself Theme-based Project. You get to decide what kind of website you want to build, as long as it adheres to the specific theme announced here: **student tools that have a social component**. This means you can create anything that satisfies this theme.

You will need to do the following:

1. Pitch your project idea to the professor, either by email or during the after-class scrum, and get the professor's approval. Then,
2. Determine the features of your website. Use the booking application non-competition, non-bonus requirements as a volume measurement for the amount of work your project will entail. Keep in mind how many teammates you have. Notice in the booking app, a team of 4 is expected to do a little more work. Then,
3. Talk to the professor again (email or after-class scrum) to get the okay on the features, given the number of teammates. This is basically a sanity check to see whether you have enough work to do or too much. Once you get the professor's approval, then you are good to go. Start

building your program.

General Expectations

You must code over 70% of your website by hand. This is important. We are computer science people; we need to know how to program. The remaining 30% can come from templates, frameworks, libraries, and LLMs.

You must reference in the **README** file the location of the 30% of your project that was not coded by yourself.

Your goal is to create a functional, useful Booking or Free-Form website that adheres to the Theme and project requirements. You are designing a professional-looking, responsive, dynamic, interactive, functional, and pretty website. It must be easy for students, professors, and TAs to use. Be creative and design not only a **good look** but also a simple, **easy-to-use** website. Simplicity of design includes **simple code**, **fewer libraries**, and **optimized functions**.

Submit the following:

- Submission 1 – Register your team – already completed in Assignment #2.
- Submission 2 – Demo & Running Program Tryout
  - **Due**: Appointment during the exam period. Cannot be later than April 30.
  - Points: **50 points, bonus 3 points if 5 days (or more) earlier than April 30**.
  - Make an appointment with the TA. Each team will be assigned a TA.
  - At this appointment, there will be two phases. In Phase 1, the TA does not touch your program, and you will demo it to them. In Phase 2, after the demonstration is complete, the TA will use the program without your help. In other words, you stay silent, and the TA will see if your program is intuitive to use and if bugs are encountered.
  - Grading: The TA will enter your grade into myCourses under "Project Demo & Tryout" for each team member. Make sure to tell the TA each team member's name. All team members receive the same grade.
- Submission 3 – Code Submission Verification
  - **Due**: Submitted to myCourses. Cannot be later than April 30.
  - Points: **50 points, bonus 2 points if earlier than April 30**.
  - Submit to myCourses under "Project Code Verification". Submit two things: (1) all source code, and (2) a README.txt file containing the URL to the running website and a table with each teammate's name and ID, plus a description of the 30% of your website not coded by your team. Each person in the team submits. In the table, list each team member and the code they worked on. The TA will check whether the team members did a similar amount of work. Add your name to the source files you programmed so the TA can run a grep on them.
  - The TA will (1) run your website again, (2) look to verify the 70% code requirement, the quality of the source code, and (3) whether the work each team member did on their own was good. **This is graded individually (each team member receives their own different grade)**.

COMPETITION INSTRUCTIONS

If you registered for the competition and then decline at the end, make sure to tell the professor and update the registration form from Assignment #2.

At the end of April, the professor will select 3 best projects. These 3 projects will be announced to the entire class via email. The professor will then submit these 3 projects to the School of Computer Science. The school reserves the right not to select any of the 3 projects. If they do select one of the projects, then an email will be sent to the class announcing the winner. This will probably happen by the end of August (maybe earlier). At the same time, the winning project will be placed on Professor Vybihal's Hall of Fame webpage. The winning project will either: (1) get adopted by the SOCS as-is!!!, or (2) go through additional development (through a COMP 400 project, a paid position, or a master's student). The winning team might be invited to help extend and deploy the website.

HOW IT WILL BE GRADED

- For the project to be graded at all, the PASS / FAIL items must all be passed.

- PASS / FAIL
  - Filled out the Registration Form.
  - Team of 3 to 4 people.
  - 70% of the project was coded from scratch.
  - Existence of frontend
  - Existence of backend
  - Existence of database
  - Each person spoke in the demo describing their section.
  - All competition projects must submit a 5-minute recording of someone using your website, with audio. This will help us when presenting the projects to the School of Computer Science.

- REDUCTIONS:
  - -20 points for not following instructions (proportional)
  - -10 points for making it difficult for the TA to grade (proportional)
  - Standard late penalty (note there are multiple due dates)

- AWARDING GRADES: The TA will award points proportionally. This means that if your feature or style is implemented only partially, you will be awarded partial points for it. For example, if you completed 25% of the feature, you would receive 25% of the points.

- DEMO & RUNNING PROGRAM TRYOUT (POINTS 50)
    - Phase 1 – Demo
        - PASS/FAIL = It runs
        - +5 = Appearance
        - +5 = Simplicity
        - +5 = Features work
    - Phase 2 – Tryout
        - +10 = Easy to use?
        - +25 = Features run well (Bugs?)

- CODE SUBMISSION VERIFICATION PER INDIVIDUAL (POINTS 50)
    - PASS/FAIL = Source code present
    - PASS/FAIL = README.txt present and filled out correctly
    - +20 = Individual code (70% code and works/bugs)
    - +20 = Quality of coding (code style, optimization, graceful execution)
    - +10 = Uses the database (CRUD, works/bugs)

Rubric

POINTS	DESCRIPTION
Full	A Level - The student answers the point exactly as described.
Partial	B Level - Solution is very good but missing 1 element. (80% points) C Level - Solution is okay, but missing 2 elements. (70% points) D Level - Solution is only partly completed. (50% points) F Level - Solution is not good. (10% to 25%)
0	Solution was not submitted, or it is completely wrong.